

Runtime View (End-to-End Flows)

Purpose

Describe the main runtime scenarios (sequence of interactions) across the Web/Mobile clients, API, database and external integrations.

Flow 1: Public Web Order (Delivery / Pickup)

Steps

1. Customer browses the menu on the **Public Ordering Web**.
2. Customer submits an order (delivery or pickup).
3. Public Web calls `POST /orders` on the **NestJS API**.
4. API validates input, computes totals, assigns initial status `CREATED`.
5. API persists the order in **PostgreSQL (RDS)**.
6. API emits an internal domain event `OrderCreated`.
7. API triggers WhatsApp adapter (optional for MVP) to send confirmation to the customer.
8. API returns `{ code: "ORDER_CREATED", data: { orderId } }` to the Public Web.
9. Customer sees confirmation message in the selected language (EN/ES) on the UI.

Key Notes

- UI displays localized messages. Backend returns stable response codes.
 - WhatsApp failures must NOT block order creation; WhatsApp is best-effort.
-

Flow 2: Internal Dine-In Order (Waiter)

Steps

1. Waiter logs in using the **Ionic Internal App**.
 2. Waiter selects a table and creates an order.
 3. Internal App calls `POST /tables/{tableId}/orders`.
 4. API validates RBAC permissions and business rules.
 5. API persists order in DB with status `CREATED`.
 6. API returns `{ code: "ORDER_CREATED" }`.
-

Flow 3: Order State Transition (Kitchen/Preparation)

Steps

1. Cashier/Admin or Waiter updates order status in Internal App.
2. Internal App calls `PATCH /orders/{orderId}/status`.
3. API validates:
 - Allowed transition (finite state machine rules)

- RBAC permissions
 - Order belongs to the restaurant context (single-tenant in MVP)
4. API stores the new status in DB.
 5. API returns `{ code: "ORDER_STATUS_UPDATED" }`.

Recommended Status Lifecycle (MVP)

- `CREATED`
 - `IN_PREPARATION`
 - `READY`
 - `OUT_FOR_DELIVERY` (delivery only)
 - `DELIVERED` (delivery only)
 - `CLOSED`
-

Flow 4: Close Sale (Cashier)

Steps

1. Cashier closes an order from Internal App.
2. Internal App calls `POST /orders/{orderId}/close`.
3. API validates:
 - Order is eligible for closure
 - Payment method is present
4. API persists closure info:
 - payment method
 - closed_at
 - closed_by
5. API (optional) creates an Integration Outbox record for Siigo:
 - status `PENDING`
6. API returns `{ code: "ORDER_CLOSED" }`.

Key Notes

- Siigo sync is async (outbox) to avoid blocking the cashier operation.
 - If Siigo is unavailable, the sale is still closed locally.
-

Flow 5: Sync Sale to Siigo (Future Adapter)

Steps

1. Background job scans Outbox records with `PENDING`.
 2. Job sends payload to Siigo via HTTPS.
 3. On success: mark as `SENT` and store external reference.
 4. On failure: mark as `FAILED`, keep error details, retry with backoff.
-

Observability (Across All Flows)

- Each request must carry a `requestId`.
- Log at least:
 - `requestId`
 - `actor (userId/role)`
 - `orderId` (when applicable)
 - `action and outcome`